

Comparison benchmark report

Same-shape measurements of this implementation and the alternatives. Blue marks our primary route.

Host `GitHub Actions 1000031976` Platform `Linux/X64` Workflow `scaling` Run `31519209588` Revision `035e615649831100a1d52626e9662b8f2a9181b7`
Measurements `75`

Direct comparisons

Blue marks our primary route. GEMM scaling charts show normalized time per MAC: flat at 1.00x is linear work scaling; rising means the implementation is losing efficiency as the problem grows.

Tropical scaling Lane scaling plus tie-dense and max-last witness scaling are included in the comparisons below.

[View the tropical charts](#)

i32 GEMM

Each row uses the same shape. The blue series is our implementation; ratios are alternative time divided by ours, so values above 1x favor ours.

SHAPE	UOR-MATMUL (OURS)	HANDWRITTEN	UOR-MATMUL SPEEDUP VS HANDWRITTEN	NDARRAY	UOR-MATMUL SPEEDUP VS NDARRAY	NALGEBRA	UOR-MATMUL SPEEDUP VS NALGEBRA
16x16x16	1.377 μ s	3.270 μ s	2.37x	2.876 μ s	2.09x	1.510 μ s	1.10x
128x128x128	148.9 μ s	3.036 ms	20.39x	2.798 ms	18.79x	488.7 μ s	3.28x
32x256x512	288.1 μ s	5.148 ms	17.87x	7.674 ms	26.63x	1.210 ms	4.20x

Relative time · logarithmic · 1x = uor-matmul (ours)

■ uor-matmul (ours) ■ alternative

16x16x16

alternative faster ←

ours	1.00x
handwritten	2.37x
ndarray	2.09x
nalgebra	1.10x

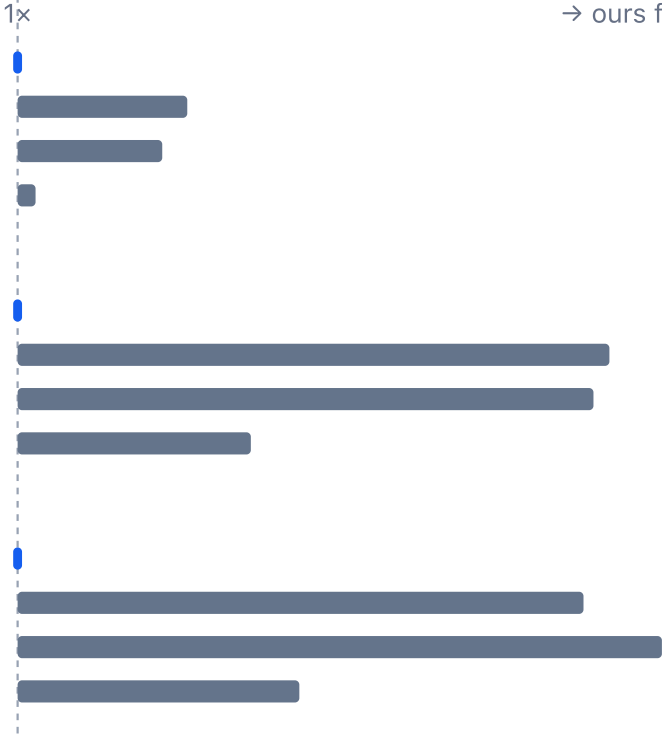
1x → ours faster

128x128x128

ours	1.00x
handwritten	20.39x
ndarray	18.79x
nalgebra	3.28x

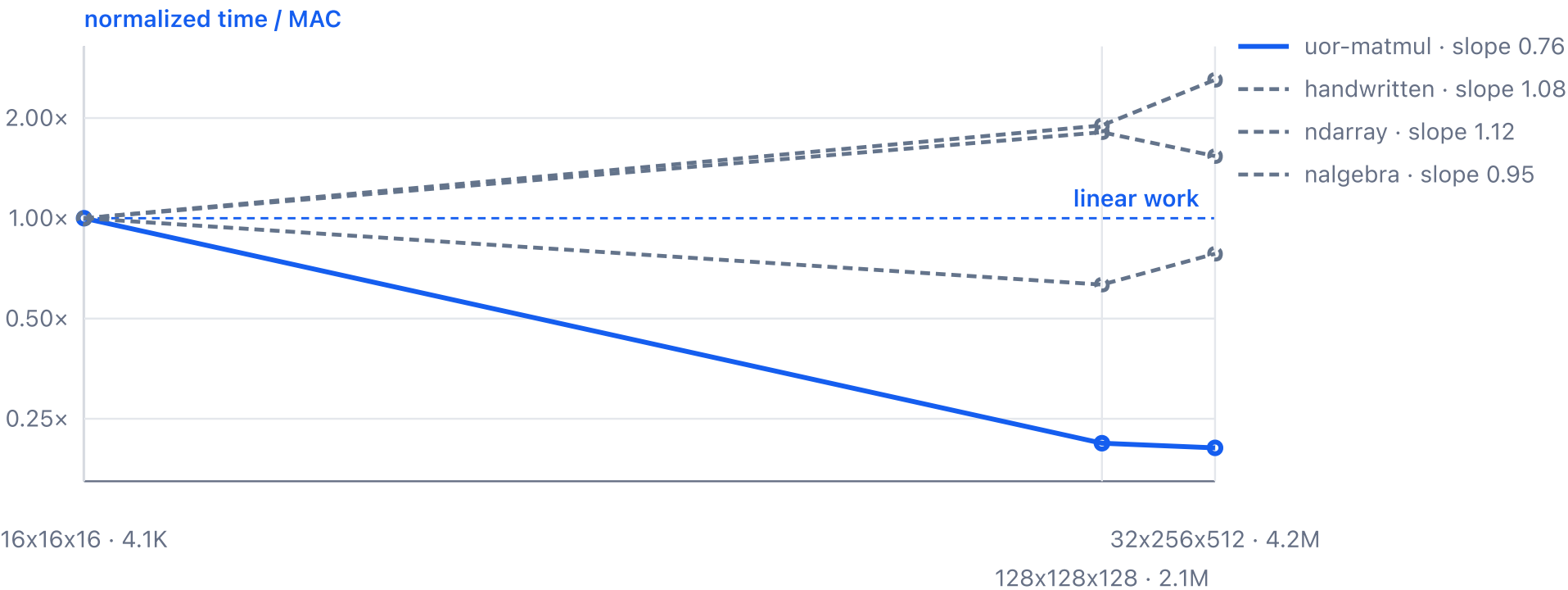
32x256x512

ours	1.00x
handwritten	17.87x
ndarray	26.63x
nalgebra	4.20x



Scaling efficiency · time per MAC, normalized to the smallest shape

A flat line means linear scaling with work. The fitted slope is time versus MAC count; 1.00 is the linear reference.



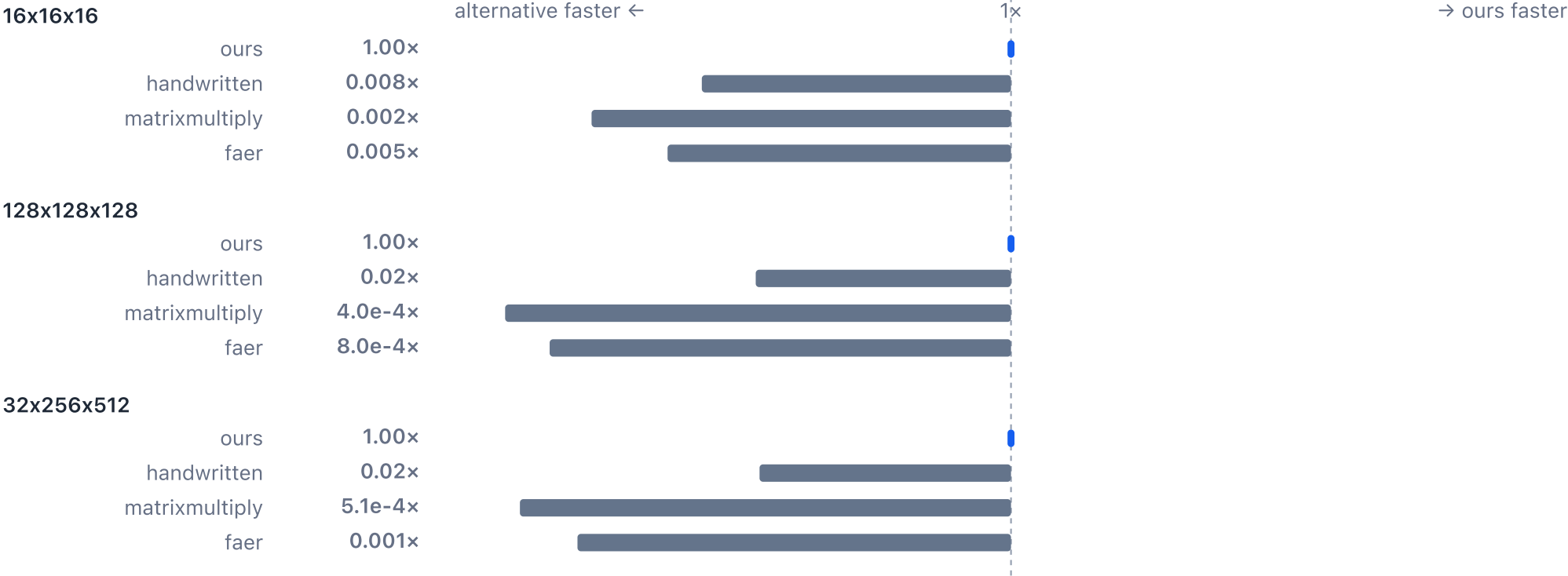
f32 GEMM

Each row uses the same shape. The blue series is our implementation; ratios are alternative time divided by ours, so values above 1x favor ours.

SHAPE	UOR-MATMUL (OURS)	HANDWRITTEN	UOR-MATMUL SPEEDUP VS HANDWRITTEN	MATRIXMULTIPLY	UOR-MATMUL SPEEDUP VS MATRIXMULTIPLY	FAER	UOR-MATMUL SPEEDUP VS FAER
16x16x16	324.6 μs	2.734 μs	0.008×	499.2 ns	0.002×	1.611 μs	0.005×
128x128x128	155.5 ms	3.014 ms	0.02×	62.86 μs	4.0e-4×	125.0 μs	8.0e-4×
32x256x512	314.6 ms	6.478 ms	0.02×	159.8 μs	5.1e-4×	388.4 μs	0.001×

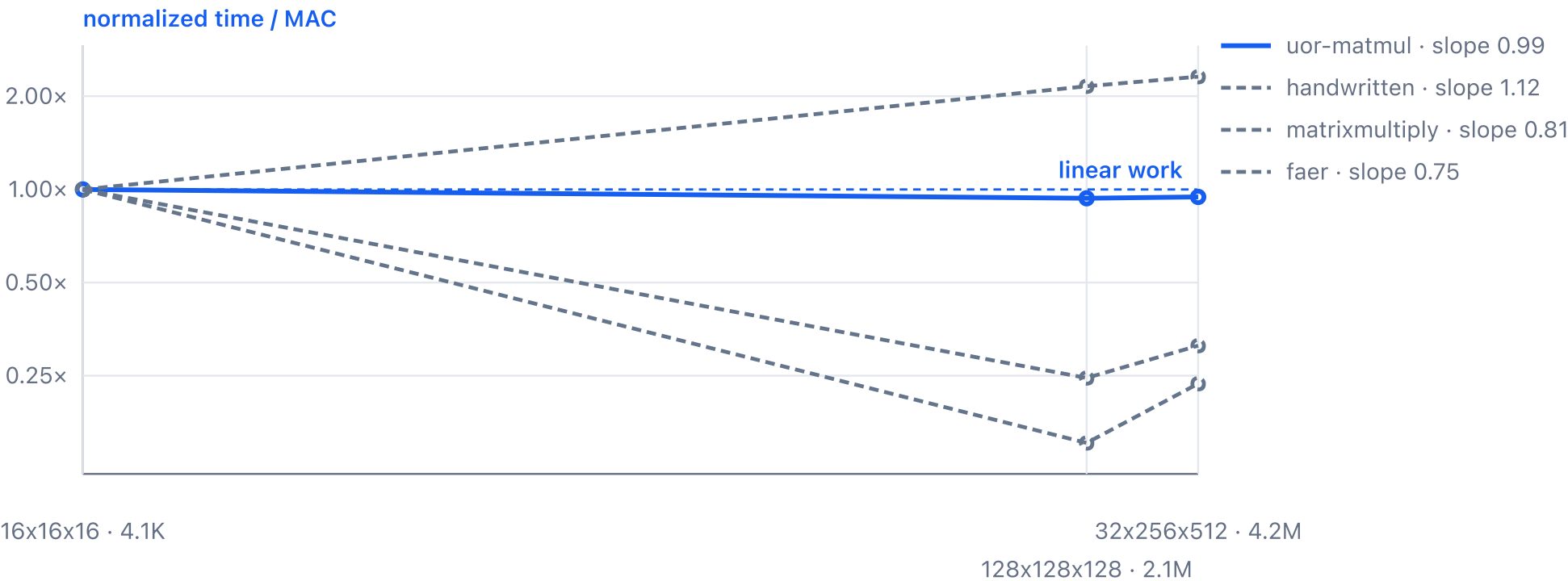
Relative time · logarithmic · 1x = uor-matmul (ours)

■ uor-matmul (ours) ■ alternative



Scaling efficiency · time per MAC, normalized to the smallest shape

A flat line means linear scaling with work. The fitted slope is time versus MAC count; 1.00 is the linear reference.



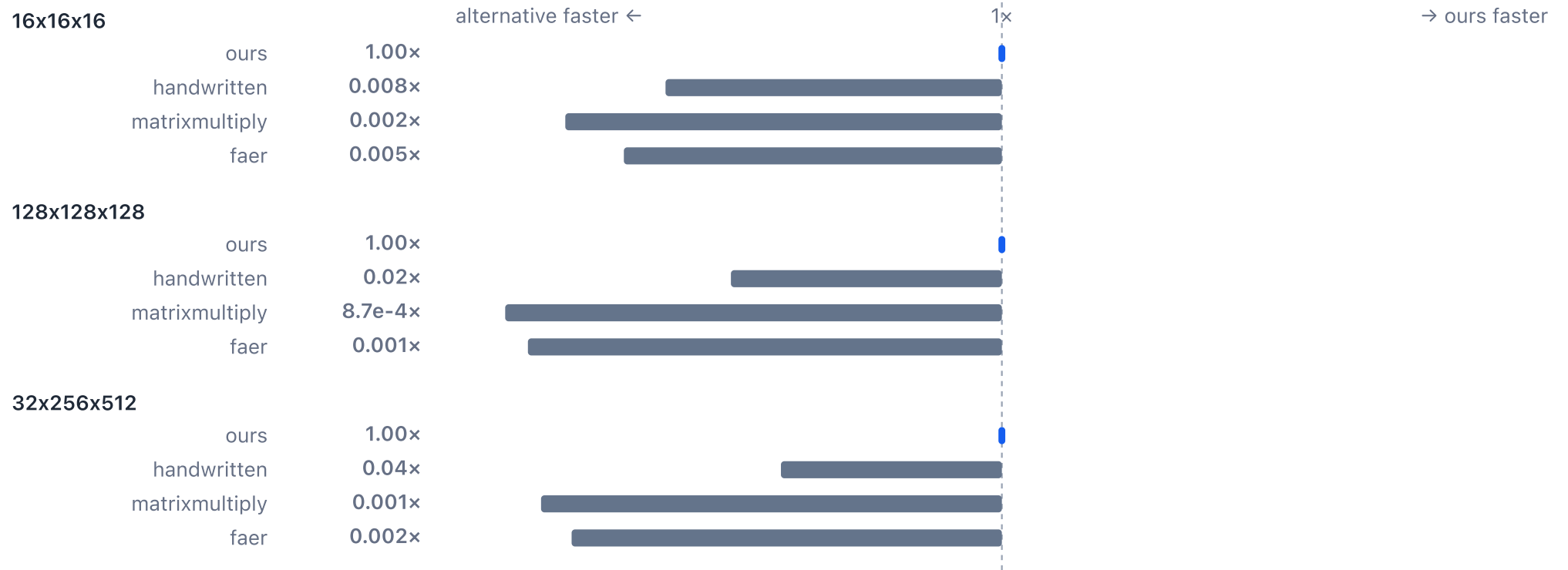
f64 GEMM

Each row uses the same shape. The blue series is our implementation; ratios are alternative time divided by ours, so values above 1x favor ours.

SHAPE	UOR-MATMUL (OURS)	HANDWRITTEN	UOR-MATMUL SPEEDUP VS HANDWRITTEN	MATRIXMULTIPLY	UOR-MATMUL SPEEDUP VS MATRIXMULTIPLY	FAER	UOR-MATMUL SPEEDUP VS FAER
16x16x16	324.3 μ s	2.734 μ s	0.008x	659.6 ns	0.002x	1.516 μ s	0.005x
128x128x128	144.8 ms	3.088 ms	0.02x	125.4 μ s	8.7e-4x	173.3 μ s	0.001x
32x256x512	289.5 ms	12.56 ms	0.04x	417.1 μ s	0.001x	644.0 μ s	0.002x

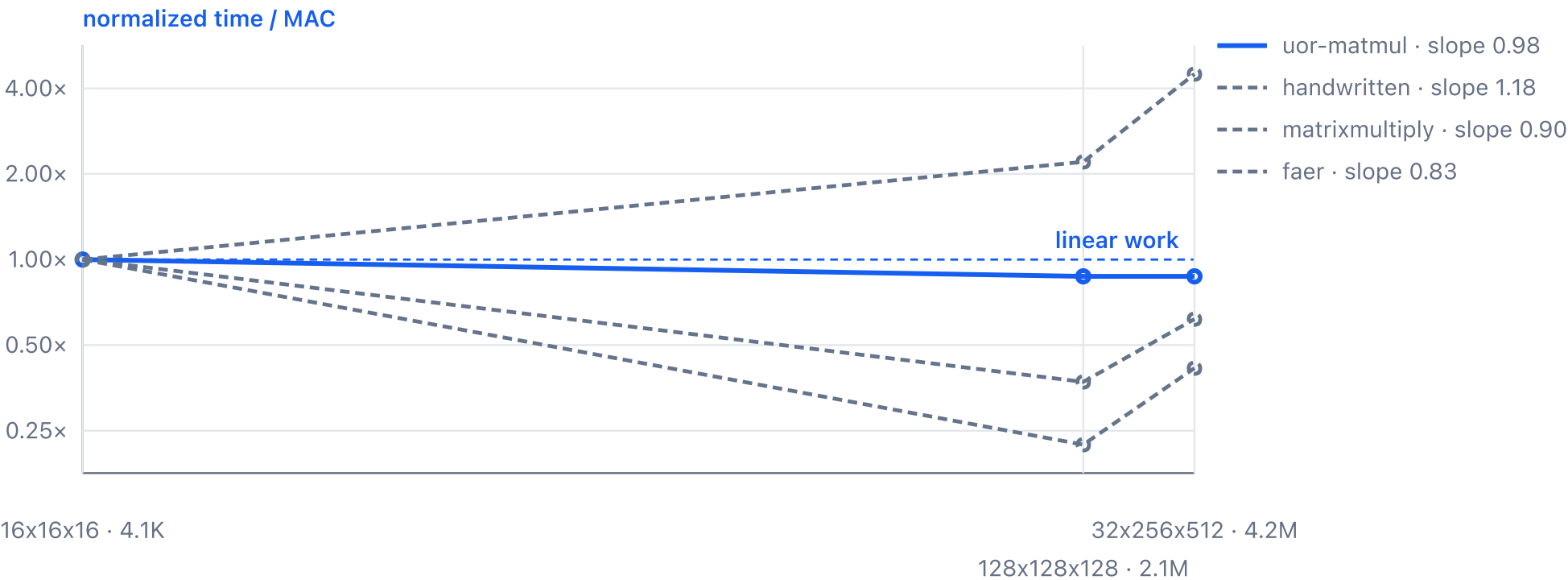
Relative time · logarithmic · 1x = uor-matmul (ours)

■ uor-matmul (ours) ■ alternative



Scaling efficiency · time per MAC, normalized to the smallest shape

A flat line means linear scaling with work. The fitted slope is time versus MAC count; 1.00 is the linear reference.

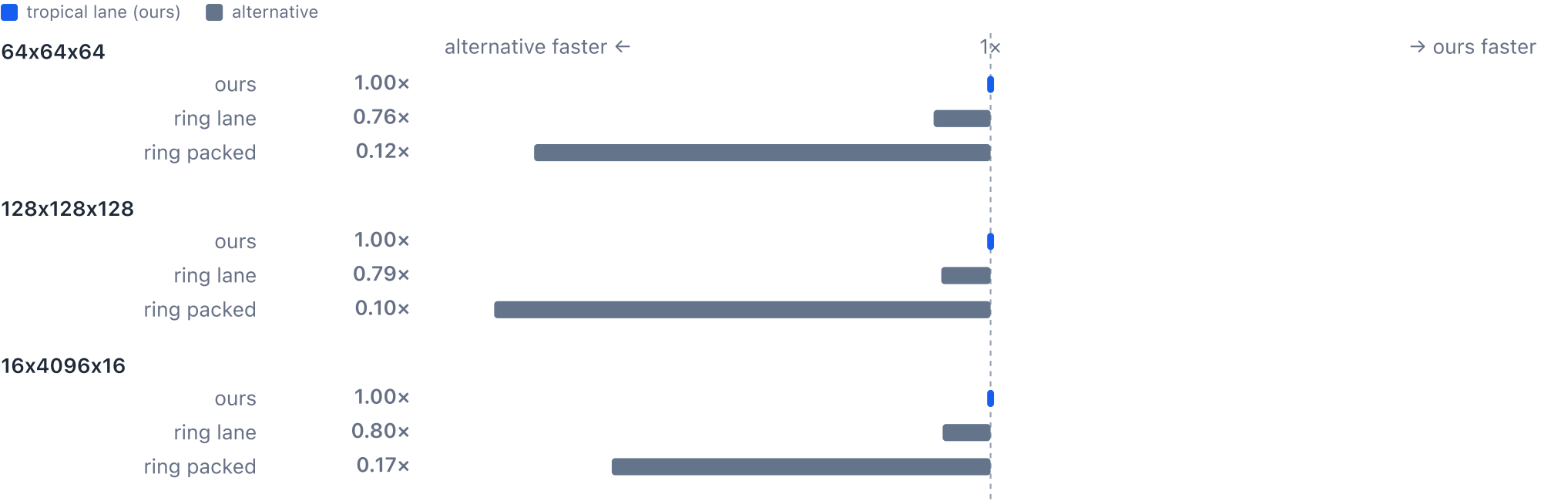


Tropical lane scaling

Each row uses the same shape. The blue series is our implementation; ratios are alternative time divided by ours, so values above 1x favor ours.

SHAPE	TROPICAL LANE (OURS)	RING LANE	TROPICAL LANE SPEEDUP VS RING LANE	RING PACKED	TROPICAL LANE SPEEDUP VS RING PACKED
64x64x64	455.7 μs	348.2 μs	0.76x	52.93 μs	0.12x
128x128x128	3.514 ms	2.784 ms	0.79x	338.0 μs	0.10x
16x4096x16	1.648 ms	1.314 ms	0.80x	276.0 μs	0.17x

Relative time · logarithmic · 1x = tropical lane (ours)



Tropical witness scaling · tie-dense

Each row uses the same shape. The blue series is our implementation; ratios are alternative time divided by ours, so values above 1x favor ours.

SHAPE	LEXICOGRAPHIC (OURS)	COMPARE-PASS	LEXICOGRAPHIC SPEEDUP VS COMPARE-PASS
tie-dense/64x64x64	998.4 μs	532.1 μs	0.53x
tie-dense/128x128x128	7.367 ms	3.574 ms	0.49x
tie-dense/16x4096x16	3.333 ms	1.447 ms	0.43x

Relative time · logarithmic · 1x = lexicographic (ours)



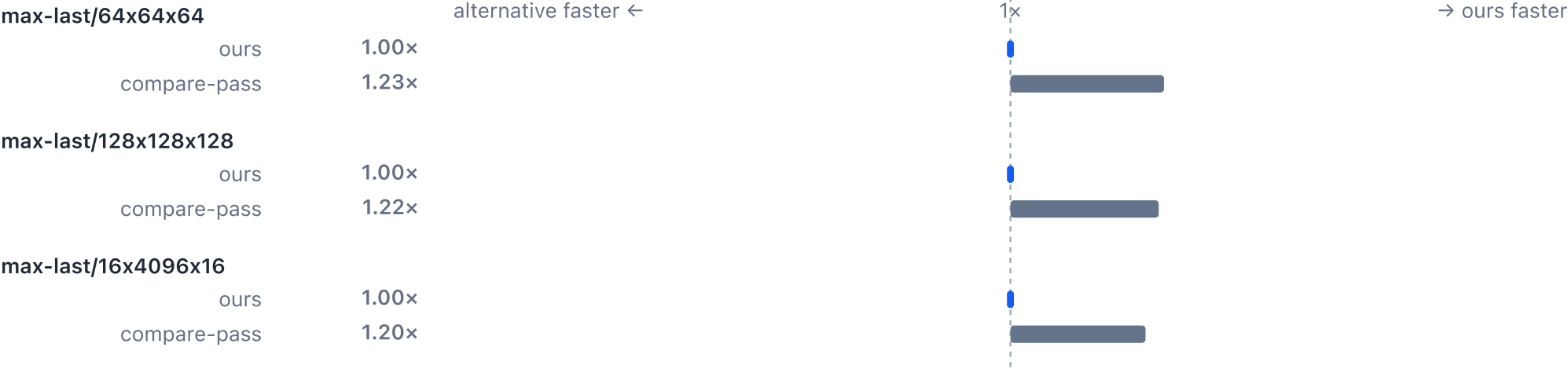
Tropical witness scaling · max-last

Each row uses the same shape. The blue series is our implementation; ratios are alternative time divided by ours, so values above 1x favor ours.

SHAPE	LEXICOGRAPHIC (OURS)	COMPARE-PASS	LEXICOGRAPHIC SPEEDUP VS COMPARE-PASS
max-last/64x64x64	741.0 μs	914.2 μs	1.23x
max-last/128x128x128	5.702 ms	6.985 ms	1.22x
max-last/16x4096x16	2.728 ms	3.283 ms	1.20x

Relative time · logarithmic · 1× = lexicographic (ours)

lexicographic (ours) alternative



Public route

Each row uses the same shape. The blue series is our implementation; ratios are alternative time divided by ours, so values above 1× favor ours.

SHAPE	SLICE::GEMM (OURS)	GEMM_PACKED	SLICE::GEMM SPEEDUP VS GEMM_PACKED
16x16x16	1.753 μs	1.770 μs	1.01×
64x64x64	52.91 μs	52.95 μs	1.00×
128x128x128	340.6 μs	339.9 μs	1.00×

Relative time · logarithmic · 1x = slice::gemm (ours)



Finite i8 lookup-table build

Each row uses the same shape. The blue series is our implementation; ratios are alternative time divided by ours, so values above 1x favor ours.

SHAPE	CPU-NATIVE (OURS)	PORTABLE	CPU-NATIVE SPEEDUP VS PORTABLE
space4096-blk16	1.009 ms	1.082 ms	1.07x

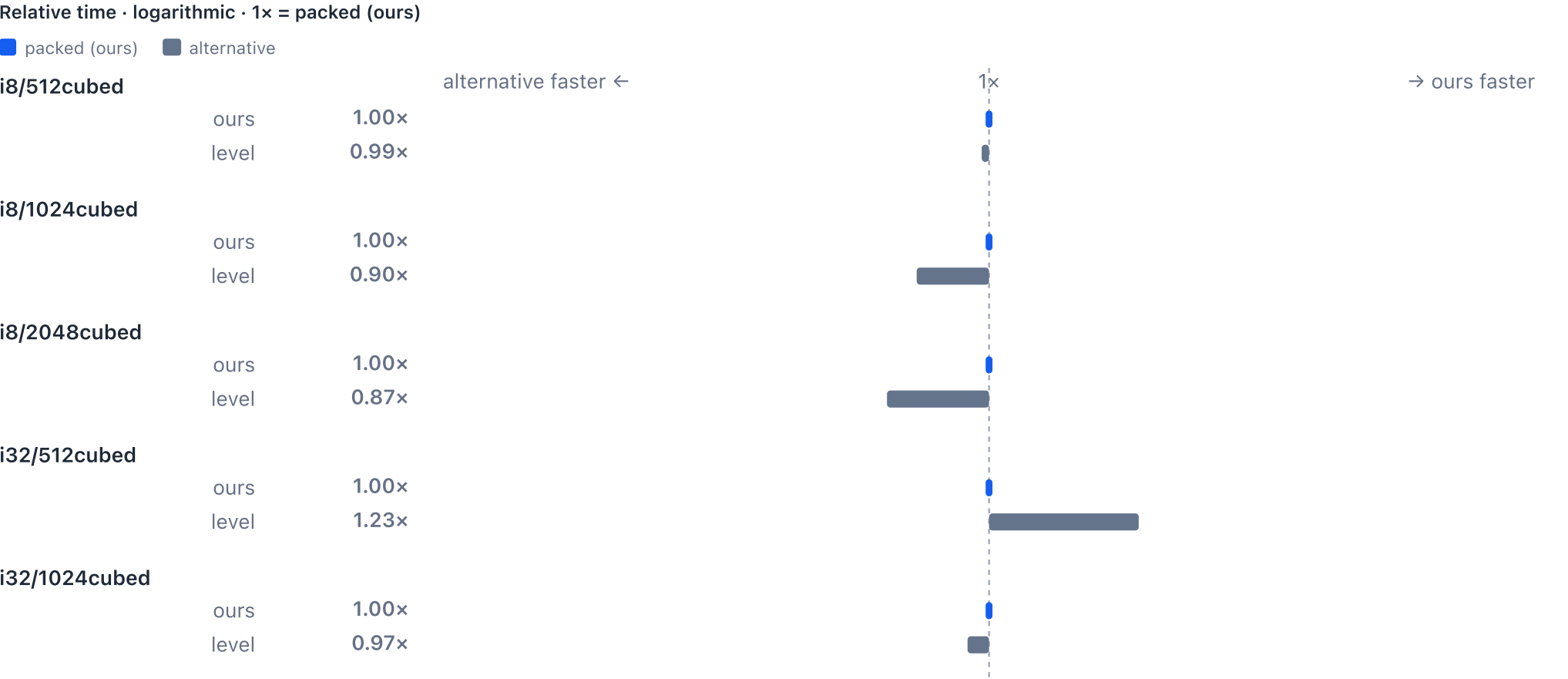
Relative time · logarithmic · 1x = cpu-native (ours)



Modular-Strassen routes

Each row uses the same shape. The blue series is our implementation; ratios are alternative time divided by ours, so values above 1x favor ours.

SHAPE	PACKED (OURS)	LEVEL	PACKED SPEEDUP VS LEVEL
i8/512cubed	16.92 ms	16.75 ms	0.99×
i8/1024cubed	135.3 ms	122.3 ms	0.90×
i8/2048cubed	1.098 s	951.9 ms	0.87×
i32/512cubed	5.109 ms	6.297 ms	1.23×
i32/1024cubed	41.60 ms	40.36 ms	0.97×



All completed measurements

GROUP	BENCHMARK	MEAN	95% CONFIDENCE INTERVAL	RAW ESTIMATE
gemm_i32	handwritten/128x128x128	3.036 ms	3.032 ms - 3.040 ms	estimates.json
gemm_i32	handwritten/16x16x16	3.270 μs	3.249 μs - 3.300 μs	estimates.json
gemm_i32	handwritten/32x256x512	5.148 ms	5.144 ms - 5.153 ms	estimates.json
gemm_i32	nalgebra/128x128x128	488.7 μs	487.6 μs - 490.5 μs	estimates.json
gemm_i32	nalgebra/16x16x16	1.510 μs	1.507 μs - 1.513 μs	estimates.json
gemm_i32	nalgebra/32x256x512	1.210 ms	1.209 ms - 1.210 ms	estimates.json
gemm_i32	ndarray/128x128x128	2.798 ms	2.791 ms - 2.804 ms	estimates.json
gemm_i32	ndarray/16x16x16	2.876 μs	2.873 μs - 2.879 μs	estimates.json
gemm_i32	ndarray/32x256x512	7.674 ms	7.672 ms - 7.676 ms	estimates.json
gemm_i32	uor-matmul/128x128x128	148.9 μs	148.6 μs - 149.4 μs	estimates.json
gemm_i32	uor-matmul/16x16x16	1.377 μs	1.375 μs - 1.380 μs	estimates.json
gemm_i32	uor-matmul/32x256x512	288.1 μs	287.6 μs - 288.9 μs	estimates.json
gemm_f32	faer/128x128x128	125.0 μs	124.7 μs - 125.3 μs	estimates.json
gemm_f32	faer/16x16x16	1.611 μs	1.610 μs - 1.612 μs	estimates.json
gemm_f32	faer/32x256x512	388.4 μs	388.1 μs - 388.7 μs	estimates.json
gemm_f32	handwritten/128x128x128	3.014 ms	3.010 ms - 3.019 ms	estimates.json
gemm_f32	handwritten/16x16x16	2.734 μs	2.723 μs - 2.745 μs	estimates.json
gemm_f32	handwritten/32x256x512	6.478 ms	6.477 ms - 6.480 ms	estimates.json
gemm_f32	matrixmultiply/128x128x128	62.86 μs	61.99 μs - 64.08 μs	estimates.json
gemm_f32	matrixmultiply/16x16x16	499.2 ns	495.9 ns - 502.1 ns	estimates.json
gemm_f32	matrixmultiply/32x256x512	159.8 μs	159.7 μs - 159.9 μs	estimates.json
gemm_f32	uor-matmul/128x128x128	155.5 ms	155.5 ms - 155.6 ms	estimates.json
gemm_f32	uor-matmul/16x16x16	324.6 μs	322.5 μs - 327.6 μs	estimates.json
gemm_f32	uor-matmul/32x256x512	314.6 ms	312.5 ms - 316.7 ms	estimates.json
gemm_f64	faer/128x128x128	173.3 μs	172.9 μs - 173.8 μs	estimates.json
gemm_f64	faer/16x16x16	1.516 μs	1.515 μs - 1.517 μs	estimates.json
gemm_f64	faer/32x256x512	644.0 μs	643.6 μs - 644.4 μs	estimates.json
gemm_f64	handwritten/128x128x128	3.088 ms	3.087 ms - 3.089 ms	estimates.json
gemm_f64	handwritten/16x16x16	2.734 μs	2.721 μs - 2.751 μs	estimates.json

GROUP	BENCHMARK	MEAN	95% CONFIDENCE INTERVAL	RAW ESTIMATE
gemm_f64	handwritten/32x256x512	12.56 ms	12.51 ms - 12.66 ms	estimates.json
gemm_f64	matrixmultiply/128x128x128	125.4 μs	125.4 μs - 125.5 μs	estimates.json
gemm_f64	matrixmultiply/16x16x16	659.6 ns	651.9 ns - 666.6 ns	estimates.json
gemm_f64	matrixmultiply/32x256x512	417.1 μs	416.5 μs - 417.6 μs	estimates.json
gemm_f64	uor-matmul/128x128x128	144.8 ms	144.6 ms - 145.1 ms	estimates.json
gemm_f64	uor-matmul/16x16x16	324.3 μs	323.9 μs - 324.8 μs	estimates.json
gemm_f64	uor-matmul/32x256x512	289.5 ms	289.4 ms - 289.7 ms	estimates.json
tropical	lane/ring-packed/128x128x128	338.0 μs	337.6 μs - 338.8 μs	estimates.json
tropical	lane/ring-packed/16x4096x16	276.0 μs	275.8 μs - 276.3 μs	estimates.json
tropical	lane/ring-packed/64x64x64	52.93 μs	52.72 μs - 53.25 μs	estimates.json
tropical	lane/ring/128x128x128	2.784 ms	2.782 ms - 2.789 ms	estimates.json
tropical	lane/ring/16x4096x16	1.314 ms	1.311 ms - 1.320 ms	estimates.json
tropical	lane/ring/64x64x64	348.2 μs	347.9 μs - 348.7 μs	estimates.json
tropical	lane/tropical/128x128x128	3.514 ms	3.498 ms - 3.541 ms	estimates.json
tropical	lane/tropical/16x4096x16	1.648 ms	1.642 ms - 1.657 ms	estimates.json
tropical	lane/tropical/64x64x64	455.7 μs	454.7 μs - 457.2 μs	estimates.json
tropical	witness/compare-pass/max-last/128x128x128	6.985 ms	6.976 ms - 6.998 ms	estimates.json
tropical	witness/compare-pass/max-last/16x4096x16	3.283 ms	3.280 ms - 3.286 ms	estimates.json
tropical	witness/compare-pass/max-last/64x64x64	914.2 μs	913.4 μs - 915.0 μs	estimates.json
tropical	witness/compare-pass/tie-dense/128x128x128	3.574 ms	3.571 ms - 3.578 ms	estimates.json
tropical	witness/compare-pass/tie-dense/16x4096x16	1.447 ms	1.443 ms - 1.453 ms	estimates.json
tropical	witness/compare-pass/tie-dense/64x64x64	532.1 μs	530.5 μs - 534.5 μs	estimates.json
tropical	witness/lexicographic/max-last/128x128x128	5.702 ms	5.698 ms - 5.710 ms	estimates.json
tropical	witness/lexicographic/max-last/16x4096x16	2.728 ms	2.726 ms - 2.732 ms	estimates.json
tropical	witness/lexicographic/max-last/64x64x64	741.0 μs	739.8 μs - 743.0 μs	estimates.json
tropical	witness/lexicographic/tie-dense/128x128x128	7.367 ms	7.356 ms - 7.386 ms	estimates.json
tropical	witness/lexicographic/tie-dense/16x4096x16	3.333 ms	3.331 ms - 3.338 ms	estimates.json
tropical	witness/lexicographic/tie-dense/64x64x64	998.4 μs	997.6 μs - 999.7 μs	estimates.json
public_api	gemm_packed/128x128x128	339.9 μs	339.5 μs - 340.8 μs	estimates.json
public_api	gemm_packed/16x16x16	1.770 μs	1.768 μs - 1.772 μs	estimates.json

GROUP	BENCHMARK	MEAN	95% CONFIDENCE INTERVAL	RAW ESTIMATE
public_api	gemm_packed/64x64x64	52.95 μs	52.87 μs - 53.04 μs	estimates.json
public_api	slice::gemm/128x128x128	340.6 μs	340.2 μs - 341.1 μs	estimates.json
public_api	slice::gemm/16x16x16	1.753 μs	1.749 μs - 1.759 μs	estimates.json
public_api	slice::gemm/64x64x64	52.91 μs	52.83 μs - 52.99 μs	estimates.json
lookup_build	build/cpu-native/space4096-blk16	1.009 ms	1.008 ms - 1.009 ms	estimates.json
lookup_build	build/portable/space4096-blk16	1.082 ms	1.081 ms - 1.082 ms	estimates.json
modular_strassen	level/i32/1024cubed	40.36 ms	40.17 ms - 40.61 ms	estimates.json
modular_strassen	level/i32/512cubed	6.297 ms	6.290 ms - 6.307 ms	estimates.json
modular_strassen	level/i8/1024cubed	122.3 ms	121.9 ms - 122.9 ms	estimates.json
modular_strassen	level/i8/2048cubed	951.9 ms	950.8 ms - 953.3 ms	estimates.json
modular_strassen	level/i8/512cubed	16.75 ms	16.74 ms - 16.76 ms	estimates.json
modular_strassen	packed/i32/1024cubed	41.60 ms	41.32 ms - 41.93 ms	estimates.json
modular_strassen	packed/i32/512cubed	5.109 ms	5.099 ms - 5.120 ms	estimates.json
modular_strassen	packed/i8/1024cubed	135.3 ms	135.1 ms - 135.5 ms	estimates.json
modular_strassen	packed/i8/2048cubed	1.098 s	1.095 s - 1.102 s	estimates.json
modular_strassen	packed/i8/512cubed	16.92 ms	16.91 ms - 16.94 ms	estimates.json